

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 21 February 2017 - Part Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY
COMMENTED

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

SIGNATURE:

(or PERSON CODE)

INSTRUCTIONS - READ CAREFULLY:

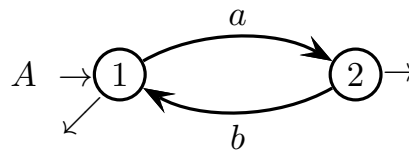
- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the following regular expression e , over the two-letter alphabet $\Sigma = \{ a, b \}$:

$$e = (a b \mid a)^+ (b a)^*$$

Consider also the finite-state automaton A below, over the same alphabet Σ , with initial state 1 and final states 1, 2:



Answer the following questions:

- (a) Find the shortest ambiguous string of the regular language $L(e)$ and suitably justify your answer. Provide evidence that the string is ambiguous and that it is the shortest ambiguous one.
 - (b) By using the Berry-Sethi method, design a finite-state deterministic automaton B that accepts the language $L(e)$, and if necessary minimize the automaton B by applying the suitable minimization algorithm.
 - (c) By using a systematic method, design a finite-state automaton C that accepts the intersection language $L(e) \cap L(A)$.
 - (d) By using the node elimination method (Brzozowsky) applied to automaton A , find an equivalent regular expression r .
-

Solution

- (a) Here is the numbered version $e_{\#}$ of the regular expression e :

$$e_{\#} = (a_1 b_2 \mid a_3)^+ (b_4 a_5)^*$$

The two numbered strings $a_1 b_2 a_3$ and $a_3 b_4 a_5$ can be derived from expression $e_{\#}$, and they correspond to the same unnumbered string $aba \in L(e)$. This proves that the regular expression e is ambiguous. It is not difficult to see that all the (infinitely many) strings $(ab)^+ a$ generated by e are ambiguous as well. Here are the strings of the language $L(e)$ that have length 1: only a ; and those that have length 2: aa , ab and ba . All such strings are non-ambiguous, as they do not have alternative derivation choices. In fact the following:

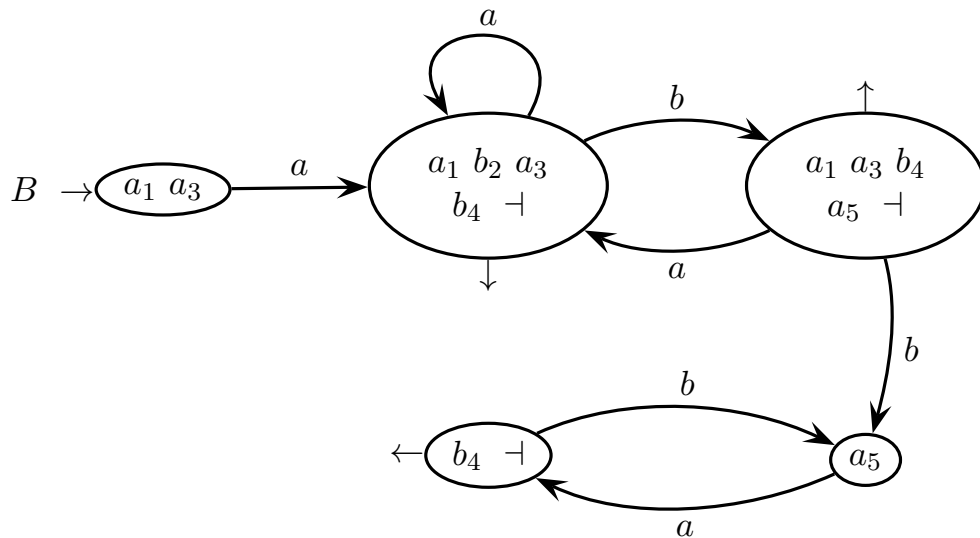
$$a : a_3 \qquad aa : a_3 a_3 \qquad ab : a_1 b_2 \qquad ba : b_4 a_5$$

are the only derivation choices possible for each of these strings. Thus string aba is the shortest ambiguous one of the regular expression e . Furthermore, it is the only ambiguous string with length 3 generated by e , since the only other string of $L(e)$ that has length 3 is $aaa : a_3 a_3 a_3$, which clearly is not ambiguous.

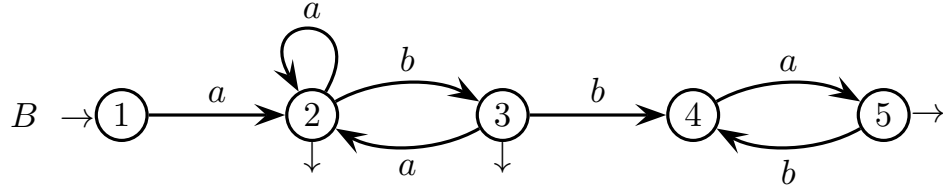
- (b) Here is the Berry-Sethi automaton B obtained from the regular expression e . First, these are the initials and followers of e :

initials	$a_1 a_3$
terminals	followers
a_1	b_2
b_2	$a_1 a_3 b_4 \dashv$
a_3	$a_1 a_3 b_4 \dashv$
b_4	a_5
a_5	$b_4 \dashv$

Then, here is the state-transition graph of automaton B produced by the *BS* algorithm, with five states:

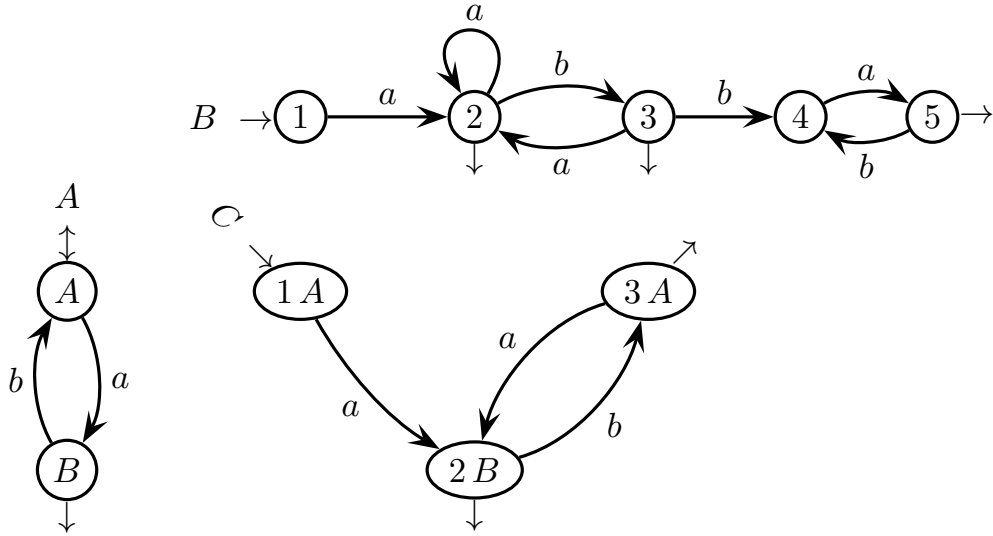


Concerning the minimality of automaton B (after a change of state names):



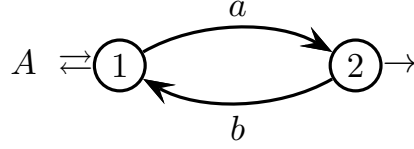
Automaton B is clean by construction (in fact all its states are accessible and post-accessible). Final states: state 5 is distinguishable from both states 2 and 3, as it does not have an outgoing a -arc; states 2 and 3 are distinguishable from each other as their outgoing b -arcs enter states 3 and 4, respectively, which are final and non-final hence distinguishable from each other. Non-final states: states 1 and 4 are distinguishable from each other, as their outgoing a -arcs enter states 2 and 5, respectively, which are already known to be distinguishable. In conclusion automaton B is in the minimal form.

- (c) From before, it holds $L(e) = L(B)$. Here is the Cartesian product of the automaton B equivalent to the regular expression e , and of the automaton A :

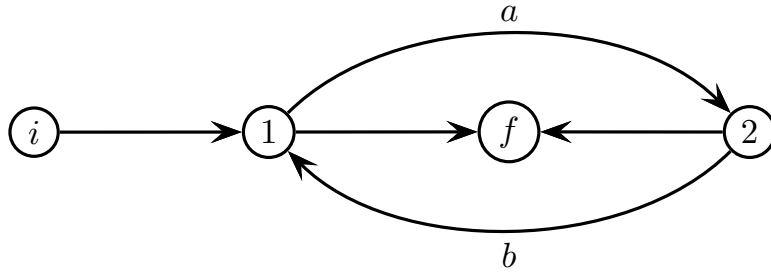


Clearly only three states of the product automaton C are both accessible (from state $1A$) and post-accessible (to states $2B$ or $3A$), whereas all the others (in principle automaton C has $2 \times 5 = 10$ states) are either inaccessible, or post-inaccessible, or both, and thus can be canceled, thus leaving a state-transition graph in the clean form. Furthermore, being the product of two deterministic automata, also automaton C is deterministic, and clearly it is already minimal, as the two final states are distinguishable since they have different outgoing arcs. Intuitively, a regular expression equivalent to the Cartesian product automaton $C = A \times B$ is $a(ba)^*[b] = (ab)^*a[b] = (ab)^+ \mid (ab)^*a$ (since it holds $a(ba)^* = (ab)^*a$). This regular expression comes from modifying the original regular expression $e = (ab \mid a)^+(ba)^*$, which is equivalent to automaton B , to make it satisfy the constraints imposed by intersecting with automaton A , namely, to have alternating letters a and b , and to have initial letter a .

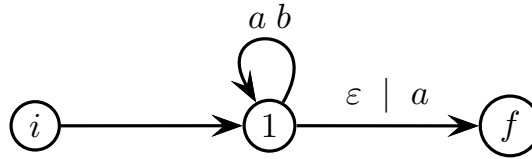
(d) Here is the node elimination (Brzozowsky) method applied to automaton A :



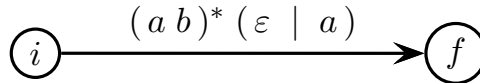
Unique initial and final states without ingoing and outgoing arcs, respectively:



Eliminate node 2:



Eliminate node 1:

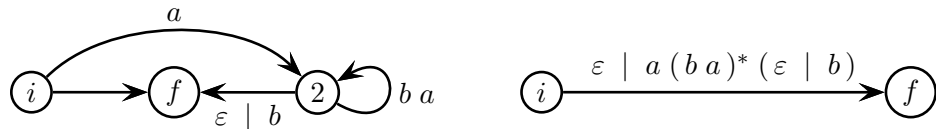


In conclusion, here is a (compact formulation of the) regular expression R :

$$R = (a b)^* (\varepsilon \mid a) = (a b)^* [a]$$

which is visibly equivalent to automaton A . Of course, the expression can be written in different equivalent forms, e.g., $R = (a b)^* \mid (a b)^* a = (a b)^* \mid a (b a)^*$, since it holds $(a b)^* a = a (b a)^*$.

If we had eliminated first node 1 then node 2, we would have obtained:



and consequently we would have had the following regular expression R' :

$$R' = \varepsilon \mid a (b a)^* (\varepsilon \mid b) = [a (b a)^* [b]]$$

equivalent to the regular expression R , though a little more complicated.

2 Free Grammars and Pushdown Automata 20%

1. Consider a two-letter alphabet $\Sigma = \{ a, b \}$ and the classical free language L of the palindromes without centre, below:

$$L = \{ x x^R \mid x \in \Sigma^* \}$$

Answer the following questions:

- (a) Write a *BNF* grammar G_1 (no matter if ambiguous) that generates the language L_1 of the (possibly empty) prefixes of language L , namely:

$$L_1 = \{ y \in \Sigma^* \mid \exists z \in \Sigma^* \text{ such that } yz \in L \}$$

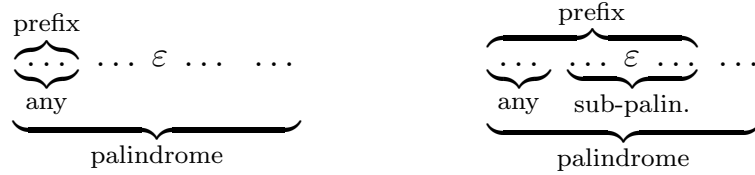
- (b) Write a *BNF* grammar G_2 (no matter if ambiguous) that generates the language L_2 of the (possibly empty) substrings of language L , namely:

$$L_2 = \{ w \in \Sigma^* \mid \exists u, v \in \Sigma^* \text{ such that } uwv \in L \}$$

- (c) (optional) Determine if grammars G_1 and G_2 are ambiguous and justify your answer: if a grammar is ambiguous then provide a string with two or more syntax trees, else argue that it does not generate any ambiguous string.
-

Solution

- (a) The question has two simple solutions, depending of how closely the structure of language L_1 is analyzed. By definition language L_1 contains the palindrome language itself, and in addition to it all the proper palindrome prefixes. Thus:
- i. A (possibly empty) prefix of a (non-centred) palindrome is any (a generic possibly empty) string followed by a (possibly empty) sub-palindrome. There are the two cases below (where ε marks the palindrome midpoint):



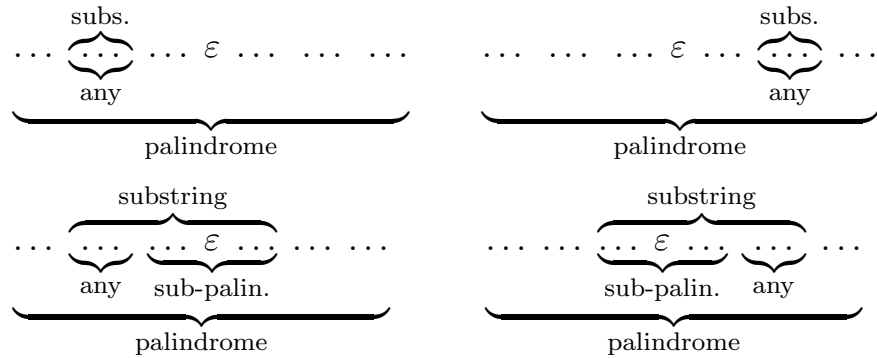
Thus a possible solution is the following grammar G_1 :

$$G_1 \begin{cases} S \rightarrow X P & \text{--- any str. \& pal.} \\ X \rightarrow a X \mid b X \mid \varepsilon & \text{--- any string} \\ P \rightarrow a P a \mid b P b \mid \varepsilon & \text{--- palindrome} \end{cases}$$

which works in all cases, since both nonterminals X and P are nullable.

- ii. Notice that a generic prefix followed by a (possibly empty) palindrome is a generic string (possibly empty), so that actually language L_1 is the universal language Σ^* . To generate language L_1 , it suffices a simplified grammar G_{univ} that only keeps the rules $S \rightarrow X$ and $X \rightarrow a X \mid b X \mid \varepsilon$ of grammar G_1 (or just the second rule with axiom X if a recursive axiom is admitted).
- (b) The situation is similar to that of question (a), namely:

- i. A (possibly empty) substring of a (non-centred) palindrome is a (possibly empty) sub-palindrome with either any (possibly empty) prefix or any (possibly empty) suffix, but not with both a prefix and a suffix. Four cases:



Thus a possible solution is the following grammar G_2 :

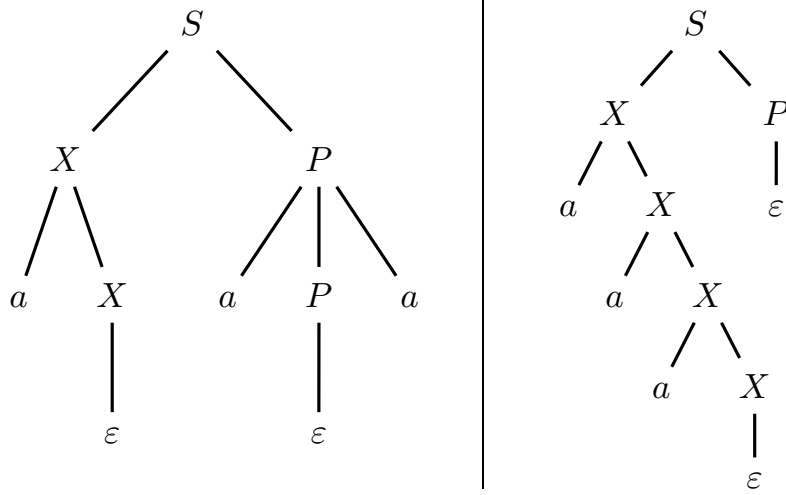
$$G_2 \begin{cases} S \rightarrow X P \mid P X & \text{--- any str. \& pal. | switch} \\ X \rightarrow a X \mid b X \mid \varepsilon & \text{--- any string} \\ P \rightarrow a P a \mid b P b \mid \varepsilon & \text{--- palindrome} \end{cases}$$

which works in all cases, since both nonterminals X and P are nullable.

- ii. Again, a generic (possibly empty) string is obtained, so that language L_2 is the universal language Σ^* as well. To generate language L_2 , it suffices the same simplified grammar G_{univ} shown before for language L_1 .

(c) Concerning ambiguity however, the solutions discussed above behave differently:

- i. Both grammars G_1 and G_2 are ambiguous. In fact string $aaa \in L_1, L_2$ is ambiguous for both grammars ! The reason is the concatenation ambiguity between the strings generated by nonterminals X and P . Here are the trees:



These trees are generated by both grammars G_1 and G_2 . They are the only trees of grammar G_1 for string aaa . Grammar G_2 has a higher ambiguity degree than G_1 and admits, in addition to the two trees above, two more trees, which are obtained by using rule $S \rightarrow PX$ instead of rule $S \rightarrow XP$.

- ii. The universal language is non-ambiguous and the simplified *BNF* grammar G_{univ} shown before, being of type $LL(1)$, is clearly non-ambiguous.

Notice that the shorter string aa is ambiguous as well (for both grammars), because both nonterminals X and P are nullable, both are able to generate aa , and consequently the axiomatic rule $S \rightarrow XP$ is affected by concatenation ambiguity. This concludes the ambiguity exam for grammars G_1 and G_2 .

The reader may be interested in modifying this exercise and let the palindrome have a centre c instead. Try to answer questions (a), (b) and (c) under such a new hypothesis. We encourage the reader to proceed by himself and take inspiration from the previous answers, independently of whether they still hold true or do not.

However, here is a possible solution with grammars G'_1 and G'_2 (axiom S), over the three-letter alphabet $\Sigma = \{a, b, c\}$ (letter c is the palindrome centre):

$$G'_1 \left\{ \begin{array}{l} S \rightarrow X \mid XP \\ X \rightarrow aX \mid bX \mid \epsilon \\ P \rightarrow aPa \mid bPb \mid c \end{array} \right. \quad G'_2 \left\{ \begin{array}{l} S \rightarrow X \mid XP \mid PX \\ X \rightarrow aX \mid bX \mid \epsilon \\ P \rightarrow aPa \mid bPb \mid c \end{array} \right.$$

The prefix language of the centred palindromes consists of arbitrary strings over the sub-alphabet $\Sigma' = \{a, b\}$, and of arbitrary strings over Σ' followed by centred palindromes. Grammar G'_1 generates these two string models.

The substring language of the centred palindromes consists of arbitrary strings over Σ' , and of centred palindromes either preceded or followed (but not both cases together) by arbitrary strings over Σ' . Grammar G'_2 generates these three string models.

Grammars G'_1 and G'_2 are inspired to grammars G_1 and G_2 , respectively: in both G'_1 and G'_2 , the palindrome rules generate at least the centre c , thus a new axiomatic alternative $S \rightarrow X$ is added, to consider the case of a prefix or of a substring that does not contain the centre c ; the other rules are unchanged (except adding the centre).

Concerning ambiguity, first consider grammar G'_1 . Observe that the sub-grammars of nonterminals X and P are unambiguous, that the sub-language $L(XP)$ generated by the concatenation XP is unambiguous (because the condition of concatenation ambiguity is not satisfied by the two sub-languages $L(X)$ and $L(P)$), and that the two sub-languages generated by X and by XP are disjoint, i.e., $L(X) \cap L(XP) = \emptyset$. Therefore grammar G'_1 is unambiguous, as it is the union of two disjoint sub-languages that are generated unambiguously (see the axiomatic rule of G'_1).

For instance, the ambiguous string aaa discussed before for the prefix language of the non-centred palindromes, now splits into three cases: aaa , $aaac$ and $acaa$. The first is a palindrome truncated before the centre c , the second is truncated immediately after the centre, and the third after the centre but before the end. This way:

$$\begin{array}{ccc} \text{prefix} & \text{prefix} & \text{prefix} \\ \underbrace{aaa \dots c \dots aaa}_{\text{palindrome}} & \underbrace{aaac \ aaa}_{\text{palindrome}} & \underbrace{aacaa}_{\text{palindrome}} \end{array}$$

None of these three cases is affected by concatenation ambiguity any more. In fact, grammar G'_1 generates each of them in these three unique ways, respectively:

$$\begin{array}{ccc} \underbrace{a \ a \ a}_X & \underbrace{a \ a \ a}_X \underbrace{c}_P & \underbrace{a}_X \underbrace{a \ c \ a}_P \end{array}$$

Of course, grammar G'_1 can still generate all the centred palindromes, as it is permitted by the definition of language L_1 , since nonterminal X is nullable and consequently all these derivations are possible: $S \Rightarrow XP \Rightarrow P \xrightarrow{*}$ any centred palindrome.

Then, considering grammar G'_2 , the same observations as for grammar G_1 apply, with only one difference: the two sub-languages $L(XP)$ and $L(PX)$ overlap, i.e., $L(XP) \cap L(PX) \neq \emptyset$. In fact, since nonterminal X is nullable, it holds $L(P) \subset L(XP)$, $L(PX)$, that is, the two sub-languages share all the centred palindromes (and only these). For instance, string c (empty palindrome) is generated by both concatenations XP and PX . Therefore grammar G'_2 is affected by union ambiguity.

Anyway, if one needs or wishes, with a little more effort it is not difficult to make grammar G'_2 unambiguous: since the sub-language overlap is caused only by non-terminal X being nullable, it suffices to make it be non-nullable, of course without changing the language. Here is a possible solution G''_2 (axiom S):

$$G''_2 \left\{ \begin{array}{ll} S \rightarrow \varepsilon \mid X \mid P \mid XP \mid PX & \text{— five substring cases} \\ X \rightarrow aX \mid bX \mid a \mid b & \text{— arbitrary string } (\neq \varepsilon) \\ P \rightarrow aPa \mid bPb \mid c & \text{— centred palindrome} \end{array} \right.$$

This grammar variant G''_2 is obtained from G'_2 by means of the standard transformation into non-nullable form (see the textbook) and is equivalent to grammar G'_2 . For instance, string c (empty palindrome) may not be generated by either concatenation XP or PX , whereas it can be generated only by nonterminal P .

2. Consider a description language that models the organizational chart of a generic institution, which is structured according to the following specifications:

- At the top level, the organization has a name (a non-empty alphanumerical string), a president, a secretary and a staff (a non-empty group of participating persons); all are mandatory.
- The organization must also include a non-empty list of committees. A committee may include one or more subcommittees, and a subcommittee may further include one or more subcommittees, at any depth level.
- A committee has a name (a non-empty alphanumerical string), a chairperson, a secretary, a staff (a non-empty group of participating persons), an establishment date and a dismissal one; all are mandatory, except for the dismissal date.
- A subcommittee has the same structure as a committee, except that the secretary and the establishment date are optional, and that a dismissal date may be specified provided that an establishment date is specified as well.
- The order of the various elements listed above in the organization, in the committees and in the subcommittees, is fixed and is shown in the example below.
- A person is identified by his first name and his last name (both names are alphabetical strings), and by his pid (personal identifier), which is a string of exactly six decimal digits. A date is composed of a day/month/year in the customary format xx/xx/xxxx, where x is a decimal digit.

Further detailed information about the lexical and syntactic structure of the description can be inferred from the example below:

```
organization CompSciConference2017
  president firstname Lilli lastname Verdi pid 839257;
  secretary firstname Paolo lastname Bianchi pid 325476;
  staff
    firstname ... lastname ... pid ...;
    firstname ... lastname ... pid ...;
  end staff
  committee communication
    chairperson firstname Mario lastname Sarti pid 937294;
    secretary firstname Lisa lastname Volo pid 739473;
    staff ... end staff
    establishment 23/07/2015;
    subcommittee invitations
      chairperson firstname Sandra lastname Piano pid 938402;
      staff ... end staff
      establishment 14/08/2015;
      dismissal 21/01/2017;
    end subcommittee invitations
  end committee communication
  ...
end organization
```

Write a grammar, *EBNF* and unambiguous, that models the sketched language.

Solution

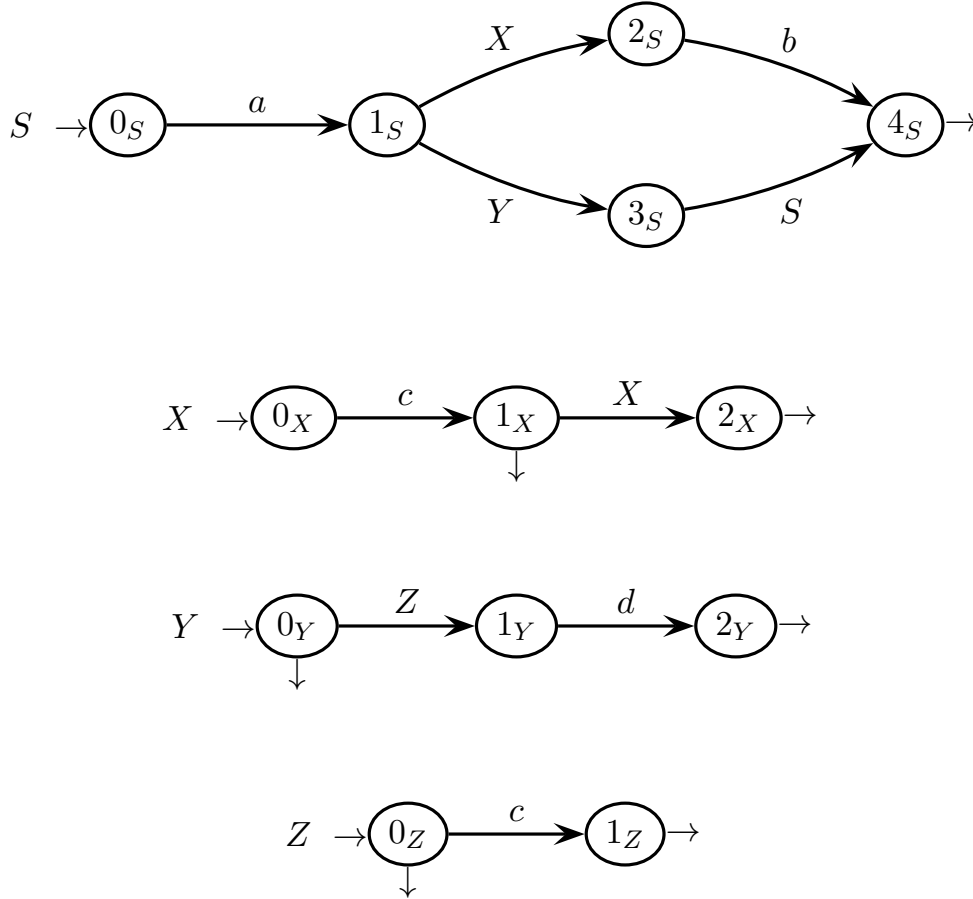
(a) Here is a reasonable grammar that generates the sketched language (axiom **ORG**):

{	$\langle \text{ORG} \rangle \rightarrow$	organization $\langle \text{STRING} \rangle$	
		president $\langle \text{PERSON} \rangle$ ‘;’	— mandatory
		secretary $\langle \text{PERSON} \rangle$ ‘;’	— mandatory
		$\langle \text{STAFF} \rangle$	— mandatory
		$\langle \text{COMM} \rangle^+$	— mand. list
		end organization	
	$\langle \text{COMM} \rangle \rightarrow$	committee $\langle \text{STRING} \rangle$	
		chairperson $\langle \text{PERSON} \rangle$ ‘;’	— mandatory
		secretary $\langle \text{PERSON} \rangle$ ‘;’	— mandatory
		$\langle \text{STAFF} \rangle$	— mandatory
		$\langle \text{PERIOD} \rangle$ ‘;’	— mandatory
		$\langle \text{SCOMM} \rangle^*$	— opt. list
		end committee $\langle \text{STRING} \rangle$	
	$\langle \text{SCOMM} \rangle \rightarrow$	subcommittee $\langle \text{STRING} \rangle$	
		chairperson $\langle \text{PERSON} \rangle$ ‘;’	— mandatory
		[secretary $\langle \text{PERSON} \rangle$ ‘;’]	— optional
		$\langle \text{STAFF} \rangle$	— mandatory
		[$\langle \text{PERIOD} \rangle$ ‘;’]	— optional
		$\langle \text{SCOMM} \rangle^*$	— opt. list
		end subcommittee $\langle \text{STRING} \rangle$	
	$\langle \text{STAFF} \rangle \rightarrow$	staff ($\langle \text{PERSON} \rangle$ ‘;’) ⁺ end staff	
	$\langle \text{PERSON} \rangle \rightarrow$	firstname $\langle \text{NAME} \rangle$ lastname $\langle \text{NAME} \rangle$ pid $\langle \text{PID} \rangle$	
	$\langle \text{PERIOD} \rangle \rightarrow$	establishment $\langle \text{DATE} \rangle$ [‘;’ dismissal $\langle \text{DATE} \rangle$]	
		— mandatory —	— optional —
	$\langle \text{STRING} \rangle \rightarrow$	($\langle \text{ALPH} \rangle$ $\langle \text{NUM} \rangle$) ⁺	— generic str.
	$\langle \text{NAME} \rangle \rightarrow$	$\langle \text{ALPH} \rangle^+$	— alphabetical
	$\langle \text{PID} \rangle \rightarrow$	$\langle \text{NUM} \rangle^6$	— pid format
	$\langle \text{DATE} \rangle \rightarrow$	$\langle \text{NUM} \rangle^2$ ‘/’ $\langle \text{NUM} \rangle^2$ ‘/’ $\langle \text{NUM} \rangle^4$	— date format
	$\langle \text{ALPH} \rangle \rightarrow$	[A...Z] [a...z]	— letters
	$\langle \text{NUM} \rangle \rightarrow$	[0...9]	— digits

The grammar is *EBNF* and unambiguous, as it composed of unambiguous constructs. The rules with similar roles are grouped, to outline the dependences between rule groups. Recursion models the inclusion of subcommittees, as their depth level is arbitrary, whereas regular constructs model all the rest. Most syntactic classes are self-explanatory and directly derive from the language specifications. However, a (less evident) class **PERIOD** models the operation time of a (sub)committee by grouping the two date fields; alternatively, they could be directly incorporated in the (sub)committee rule, mandatorily or optionally. Identifiers and numbers are modeled as customary. Minor adjustments are possible, e.g., to better model alphanumerical strings or to differently place semicolon separators, etc. Other solutions are also possible, more or less different.

3 Syntax Analysis and Parsing Methodologies 20%

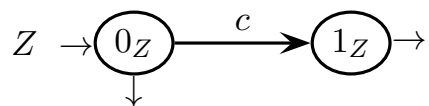
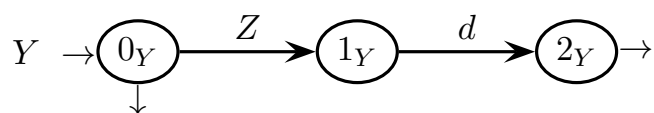
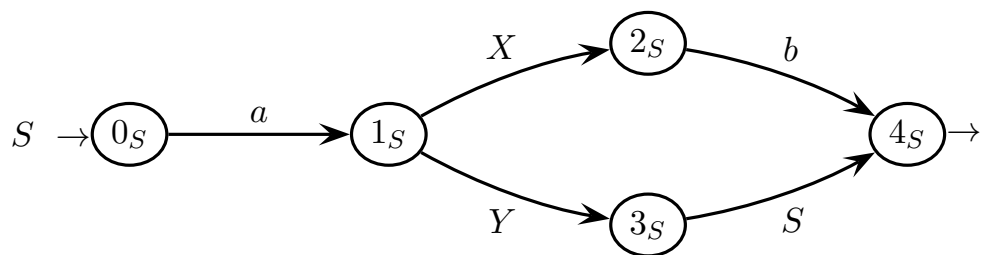
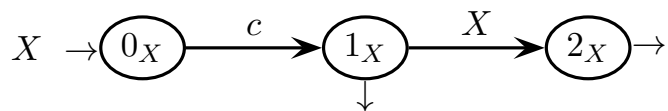
1. Consider the following grammar G , represented as a machine net over the four-letter terminal alphabet $\Sigma = \{ a, b, c, d \}$ and the four-letter nonterminal alphabet $V = \{ S, X, Y, Z \}$ (axiom S):



Answer the following questions:

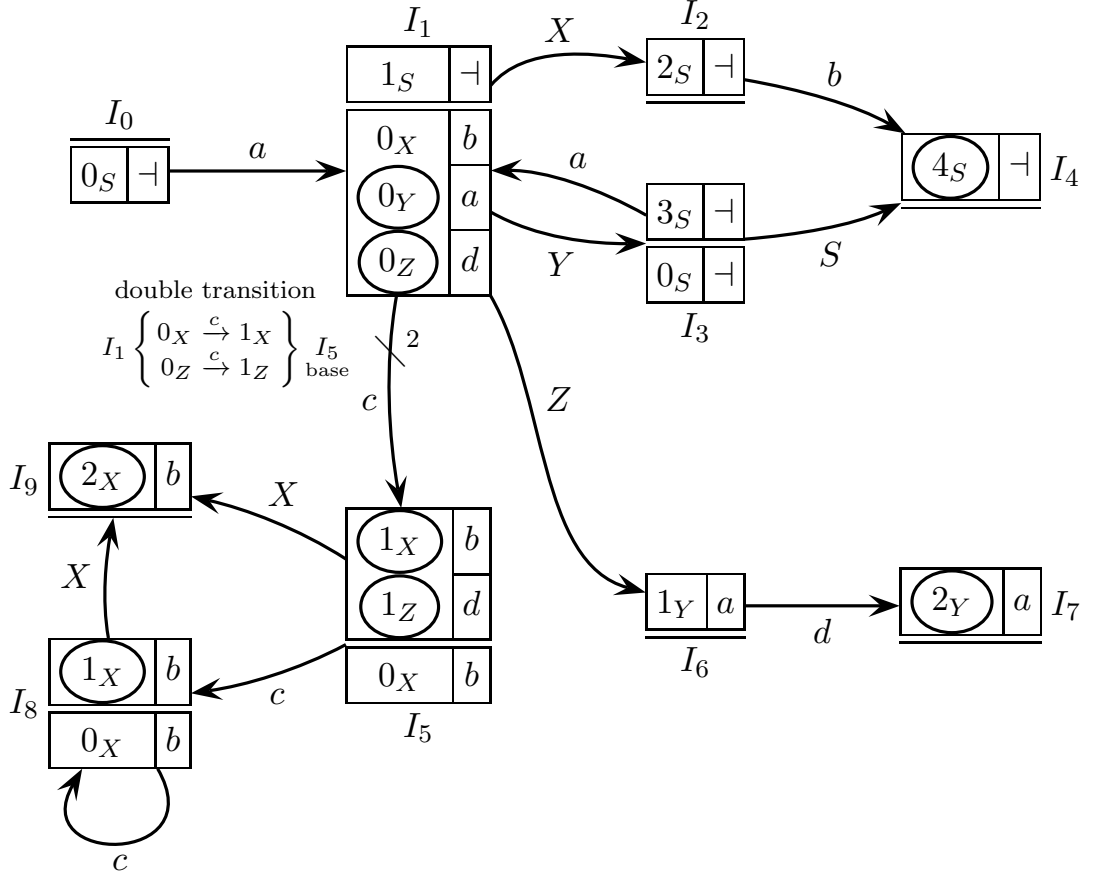
- (a) Draw the complete pilot of grammar G , say if grammar G is of type $ELR(1)$, and shortly justify your answer. If grammar G is not $ELR(1)$ then highlight all the conflicts in the pilot.
- (b) Write the necessary guide sets on the arcs of the machine net, show that grammar G is not of type $ELL(1)$, based on the guide sets, and shortly justify your answer (for the guide sets please use the figure prepared on the next page).
- (c) Say if grammar G is of type $ELL(k)$ for some $k > 1$, and provide an adequate justification to you answer.
- (d) (optional) Identify the four languages $L(Z)$, $L(Y)$, $L(X)$ and $L(S)$ by resorting, for representing each of them, to a formalism as simple as possible. Determine if there is a grammar G' equivalent to grammar G that is of type $ELL(1)$, and adequately justify your answer.

please here draw the call arcs and write the guide sets with $k = 1$



Solution

(a) Here is the pilot of grammar G , with ten m-states (I_0 is the initial m-state):

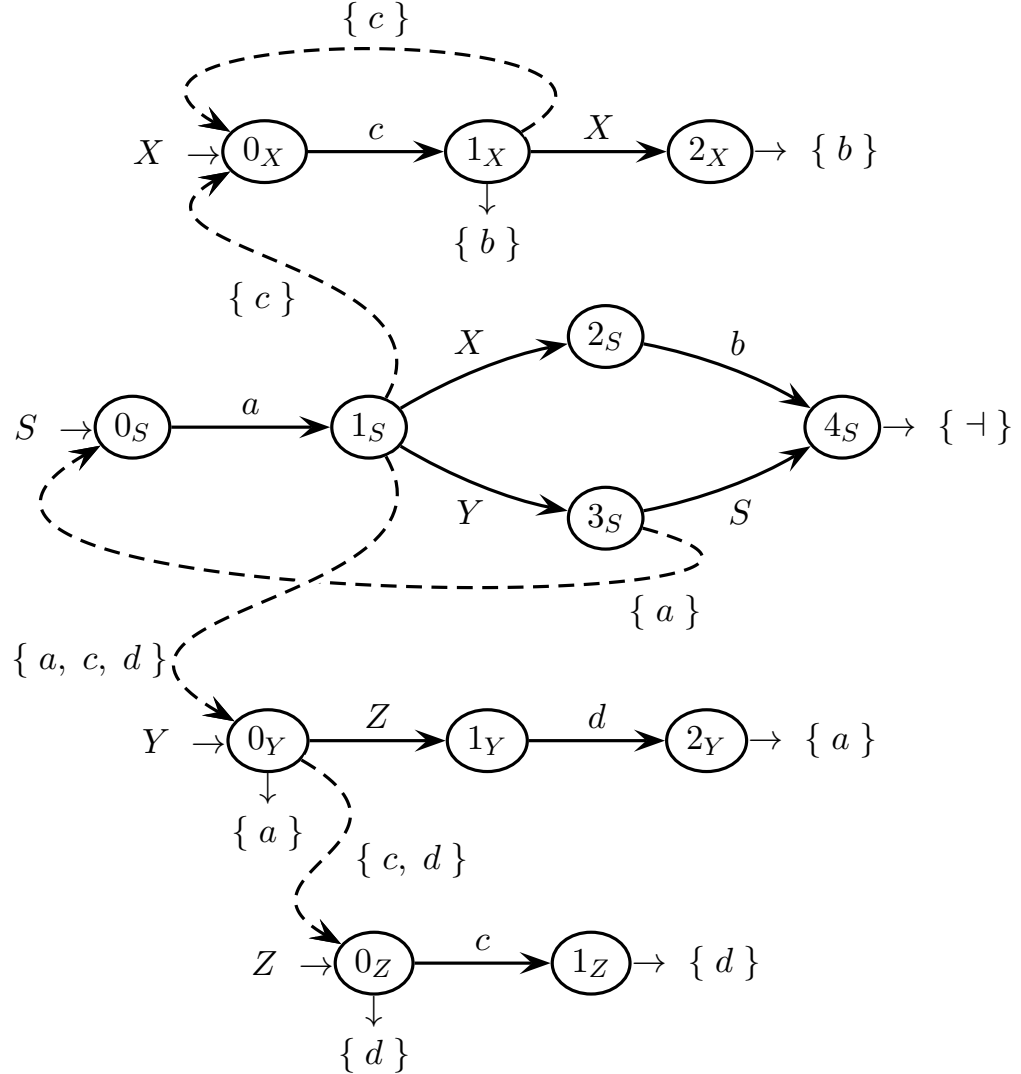


The pilot contains a few reduction items, and each of the m-states I_1 and I_5 contains two reduction items, yet there are not any shift-reduce or reduce-reduce conflicts. Furthermore, the only multiple (precisely double) transition is $I_1 \xrightarrow{c} I_5$, yet it is not convergent, so there are not any convergence conflicts either. Thus the pilot is conflict-free and consequently grammar G is of type $ELR(1)$.

Notice that the pilot does not have the STP , since transition $I_1 \xrightarrow{c} I_5$ is multiple or, equivalently, the base of m-state I_5 contains two items. This is the only aspect that causes the pilot to lose the STP . Of course, failing to have the STP immediately makes grammar G fail to be of type $ELL(1)$ ¹. Anyway, though the $ELL(1)$ condition allows us to quickly determine whether grammar G is of type $ELL(1)$, it does not give us more insight, and for better understanding where and why the $ELL(1)$ property fails in the grammar G , see point (b).

¹The other two requirements of the $ELL(1)$ condition — that the grammar must be of type $ELR(1)$ and that it must not be left-recursive — here would be satisfied; yet the third one — that the grammar must have the STP — here is not satisfied, thus the grammar is not of type $ELL(1)$.

- (b) Here are the guide sets of grammar G , with $k = 1$ (those on the terminal shift arcs are understood):



Computing the guide sets is simple. The final guide (prospect) sets are straightforward: nonterminals Z , Y , X and S are followed by the look-ahead terminals d , a , b and $\neg$, respectively. Nonterminals X and S are recursive, yet they occur at the end of their machines, thus they do not inherit any other look-ahead.

The guide set on the call arc $1_S \xrightarrow{\{a, c, d\}} 0_Y$ contains terminal a because nonterminal Y is nullable and a is an initial of nonterminal S , which follows Y . The reason is *not* that terminal a is contained in the guide (prospect) set of the exit arrow $0_Y \rightarrow$. A similar argument applies to the call arc $0_Y \xrightarrow{\{c, d\}} 0_Z$ as for terminal d . The guide set of $1_S \xrightarrow{\{a, c, d\}} 0_Y$ also contains terminal d because nonterminal Z is nullable, while the presence of terminal c is obvious herein as well as in $0_Y \xrightarrow{\{c, d\}} 0_Z$. All the remaining guide sets are obvious, too.

On the bifurcation state 1_S , the guide sets of the two call arcs are not disjoint, thus grammar G is not of type $ELL(1)$. All the other guide sets on bifurcation states, namely on the states 1_X , 0_Y and 0_Z , are disjoint.

- (c) The only conflicting state is 1_S , and the only overlapping element in the guide sets from such a state is letter c . So we have to move to a look-ahead distance $k = 2$, but we can limit ourselves to considering only such crucial element. We have to examine the possible move sequences of the predictive parser (call, shift and return moves) that start from state 1_S and contain two shifts, such that the first shift is on letter c , while the second shift has to be determined.

The guide set on the call arc $1_S \dashrightarrow 0_X$ becomes $\{cb, cc\}$. In fact, the predictive parser can perform only these two move sequences that read first letter c and then one more letter in the input tape:

$$\begin{aligned} 1_S &\xrightarrow[\text{call}]{\quad} 0_X \xrightarrow[\text{shift}]{c} 1_X \xrightarrow[\text{return}]{\quad} 2_S \xrightarrow[\text{shift}]{b} 4_S \\ 1_S &\xrightarrow[\text{call}]{\quad} 0_X \xrightarrow[\text{shift}]{c} 1_X \xrightarrow[\text{call}]{\quad} 0_X \xrightarrow[\text{shift}]{c} 1_X \end{aligned}$$

Instead, the guide set on the call arc $1_S \dashrightarrow 0_Y$ becomes $\{a, cd, d\}$. In fact, the predictive parser can perform only this move sequence that reads first letter c and then one more letter in the input tape:

$$1_S \xrightarrow[\text{call}]{\quad} 0_Y \xrightarrow[\text{call}]{\quad} 0_Z \xrightarrow[\text{shift}]{c} 1_Z \xrightarrow[\text{return}]{\quad} 1_Y \xrightarrow[\text{shift}]{d} 2_Y$$

The other elements a and d of the guide set do not require to be extended to $k = 2$, as they do not take any part in the overlap of the two guide sets. However, for completeness, such elements extend in this way: aa, ac, ad , and da (the reader may wish to verify and find the corresponding move sequences).

Thus now with look-ahead distance $k = 2$ the guide sets on the two call arcs from state 1_S are disjoint, therefore grammar G is of type $ELL(2)$.

For completeness, the element extensions mentioned above are the following:

$$1_S \xrightarrow[\text{call}]{\quad} 0_Y \xrightarrow[\text{return}]{\quad} 3_S \xrightarrow[\text{call}]{\quad} 0_S \xrightarrow[\text{shift}]{a} 1_S \xrightarrow[\text{call}]{\quad} \dots \xrightarrow[\text{return}]{\quad} 0_S \xrightarrow[\text{shift}]{a} 1_S$$

that yields extension aa , and

$$\begin{aligned} 1_S &\xrightarrow[\text{call}]{\quad} 0_Y \xrightarrow[\text{return}]{\quad} 3_S \xrightarrow[\text{call}]{\quad} 0_S \xrightarrow[\text{shift}]{a} 1_S \xrightarrow[\text{call}]{\quad} 0_Y \xrightarrow[\text{call}]{\quad} 0_Z \\ 0_Z &\xrightarrow[\text{shift}]{c} 1_Z \end{aligned}$$

that yields extension ac , and

$$\begin{aligned} 1_S &\xrightarrow[\text{call}]{\quad} 0_Y \xrightarrow[\text{return}]{\quad} 3_S \xrightarrow[\text{call}]{\quad} 0_S \xrightarrow[\text{shift}]{a} 1_S \xrightarrow[\text{call}]{\quad} 0_Y \xrightarrow[\text{call}]{\quad} 0_Z \\ 0_Z &\xrightarrow[\text{return}]{\quad} 1_Y \xrightarrow[\text{shift}]{d} 2_Y \end{aligned}$$

that yields extension ad , and finally

$$\begin{aligned} 1_S &\xrightarrow[\text{call}]{\quad} 0_Y \xrightarrow[\text{call}]{\quad} 0_Z \xrightarrow[\text{return}]{\quad} 1_Y \xrightarrow[\text{shift}]{d} 2_Y \xrightarrow[\text{return}]{\quad} 3_S \xrightarrow[\text{call}]{\quad} 0_S \\ 0_S &\xrightarrow[\text{shift}]{a} 1_S \end{aligned}$$

that yields extension da . Yet all these are unnecessary for predictive analysis.

- (d) It turns out that all the four languages are finite or at worst regular. In fact, the following language representations hold (using the optionality operator $[]$):

$$L(Z) = \varepsilon \mid c = [c]$$

due to direct evidence. Then:

$$L(Y) = \varepsilon \mid L(Z) d = \varepsilon \mid (\overbrace{\varepsilon \mid c}^{L(Z)}) d = \varepsilon \mid [c] d = [[c] d]$$

due to substitution. Then:

$$L(X) = c (\varepsilon \mid L(X)) = c \mid c L(X) = c c^* = c^+$$

due to the Arden identity. Then:

$$\begin{aligned} L(S) &= a (L(X) b \mid L(Y) L(S)) \\ &= a L(X) b \mid a L(Y) L(S) \\ &= (a L(Y))^* a L(X) b \end{aligned}$$

due to the Arden identity again. Eventually:

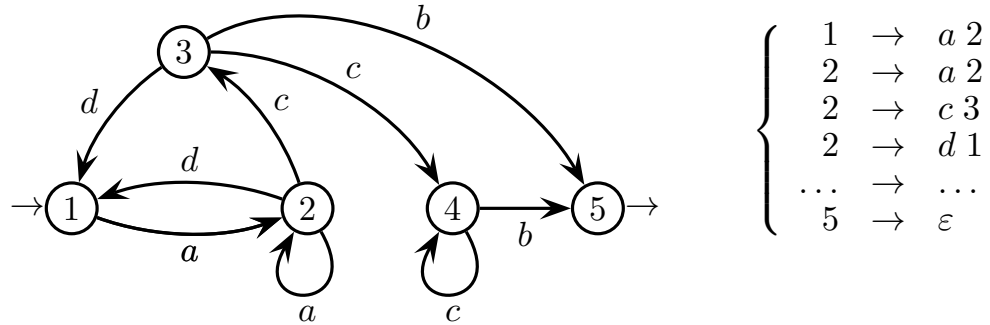
$$L(S) = \left(a \underbrace{(\varepsilon \mid (\varepsilon \mid c) d)}_{L(Y)} \right)^* a \underbrace{c^+}_{L(X)} b$$

due to substitution again. This can be written in a slightly more compact form:

$$L(S) = \left(a [[c] d] \right)^* a c^+ b$$

that also uses the optionality operator $[]$. In all the four cases the simplest formalism to represent each language is a regular expression. Actually languages $L(Z)$ and $L(Y)$ are even finite, and we could just list their strings: $L(Z) = \{ \varepsilon, c \}$ and $L(Y) = \{ \varepsilon, d, cd \}$; instead, languages $L(X)$ and $L(S)$ are infinite. Since by definition $L(G) = L(S)$, grammar G generates a regular language. Consequently, it is well known that there is a finite-state deterministic automaton equivalent to G , and that the right-unilinear grammar associated to such an automaton is of type $ELL(1)$, precisely due to determinism. Thus language $L(G)$ is of type $ELL(1)$, and a viable $ELL(1)$ (actually $LL(1)$) grammar for $L(G)$ is this (*BNF*) right-unilinear one (the reader may wish to find it for practice).

For instance, the minimal deterministic automaton equivalent to grammar G is the following (obtainable with the Berry-Sethi method and minimization):



On the right there is (part of) the corresponding right-unilinear *BNF* grammar (axiom 1), which is of type $LL(1)$ (the reader may complete it by himself).

4 Language Translation and Semantic Analysis 20%

1. The grammar G below (axiom S) generates bit strings that represent positive natural numbers according to the usual positional binary encoding:

$$G \left\{ \begin{array}{l} S \rightarrow S0 \mid S1 \mid N1 \\ N \rightarrow N0 \mid N1 \mid \varepsilon \end{array} \right.$$

For instance, string 010100 represents in binary the (decimal) number 20, string 1010 represents number 10 and string 011 number 3.

Please answer the following questions:

- (a) Define a translation scheme that concatenates to the source string a prefix consisting of a number of symbols ‘ e ’ equal to the exponent of the largest power of 2 of which the encoded number is a multiple.

For instance, the string 010100 encodes number 20, which is a multiple of 4 (but not a multiple of 8), thus it is translated into string $ee010100$. Other examples are $1010 \mapsto e1010$ and $011 \mapsto 011$. If necessary, modify the source grammar as little as possible.

Can this translation be modeled with a rational (regular) translation scheme ? In the negative case, please justify your answer; in the positive case, provide such a regular translation scheme.

- (b) Define a translation scheme that cancels all the leading zeroes from the source string. For instance: $010100 \mapsto 10100$, $1010 \mapsto 1010$ and $011 \mapsto 11$. If necessary, modify the source grammar as little as possible.

Can this translation be modeled with a rational (regular) translation scheme ? In the negative case, please justify your answer; in the positive case, provide such a regular translation scheme.

- (c) Is the translation of the previous point (b) surjective ? Is it injective ? To answer these questions, consider the source and destination languages as domain and image of the translation function, respectively. Provide an adequate justification to your answer.

Solution

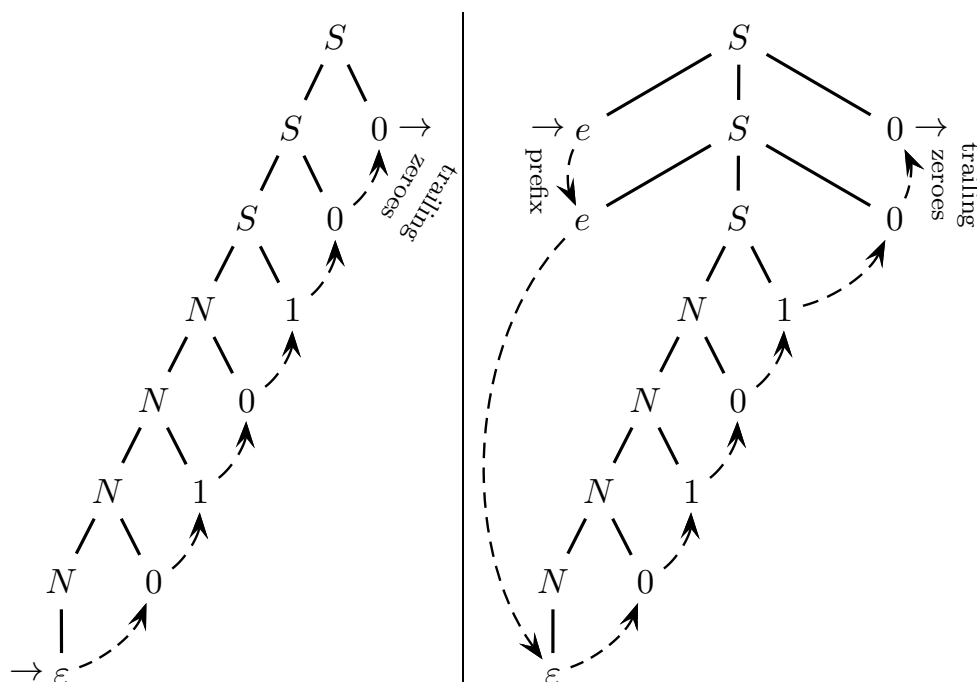
- (a) The translation counts how many trailing zeroes the source string has, and it prepends as many characters e to the destination string. We have to separate the generation of the trailing zeroes from those that are not trailing, thus grammar G has to be modified. Here is a modified source grammar G'_s (axiom S):

$$G'_s \left\{ \begin{array}{l} S \rightarrow S0 \mid N1 \\ N \rightarrow N0 \mid N1 \mid \varepsilon \end{array} \right.$$

where the rule $S \rightarrow S1$ has been removed to let nonterminal S generate all and only the trailing zeroes. Here is the corresponding destination grammar G_d' :

$$G'_d \left\{ \begin{array}{l} S \rightarrow e S 0 \mid N 1 \\ N \rightarrow N 0 \mid N 1 \mid \varepsilon \end{array} \right.$$

which prepends exactly one character e for each trailing zero, and only for those. The source and destination syntax trees for the longest example before should convince anybody of the correctness of the translation scheme G' :



The dashed arrows trace the frontiers of the source and destination trees, and show the insertion of the prefix into the destination one.

The translation cannot be modeled by means of a regular translation scheme, because it requires to *count* the number of trailing zeroes. Said differently, the destination language is $e^n (0 \mid 1)^* 1 0^n$, with $n \geq 0$, and it is not regular.

Incidentally, notice that the original source grammar G is ambiguous, as a string of bits 1 can be generated by repeatedly using either rule $S \rightarrow S\ 1$ or rule $N \rightarrow N\ 1$, or a mix thereof. Instead, the modified source grammar G'_s is not ambiguous any longer, because rule $S \rightarrow S\ 1$ has been removed.

- (b) For this second translation, we can take as basis the scheme G' , which already generates the trailing zeroes only by nonterminal S , and just switch the trailing zeroes into leading zeroes. For this purpose, it suffices to turn the alternative rules that expand nonterminal S from the left-linear form into the right-linear form (those of N can stay). Here is a modified source grammar G''_s (axiom S):

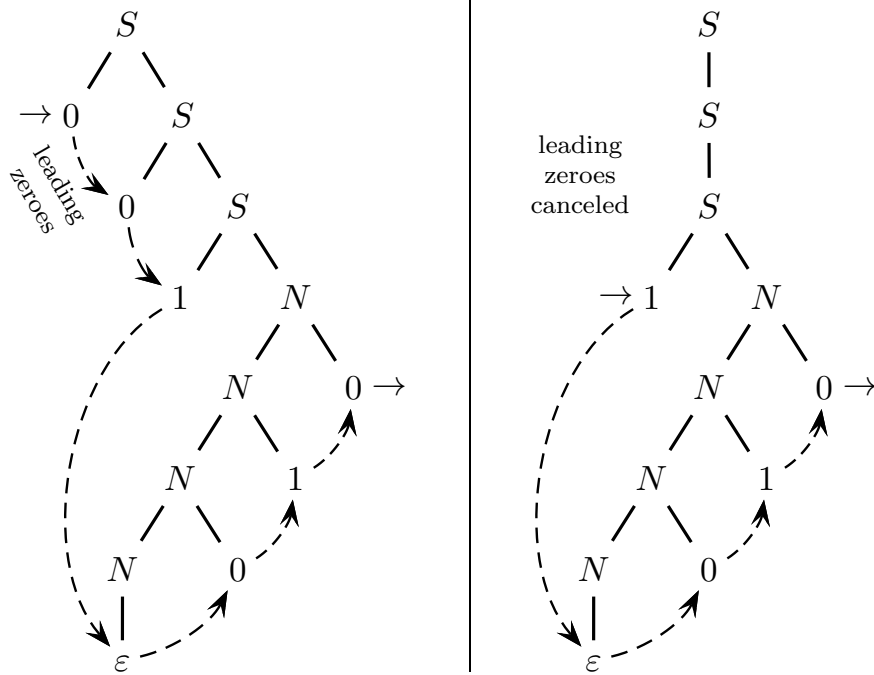
$$G''_s \left\{ \begin{array}{l} S \rightarrow 0 S \mid 1 N \\ N \rightarrow N 0 \mid N 1 \mid \varepsilon \end{array} \right.$$

where nonterminal S generates all and only the leading zeroes plus the leftmost bit 1. Here is the corresponding destination grammar G''_d :

$$G''_d \left\{ \begin{array}{l} S \rightarrow S \mid 1 N \\ N \rightarrow N 0 \mid N 1 \mid \varepsilon \end{array} \right.$$

which just cancels the leading zeroes, and only those.

Again, the source and destination syntax trees for the longest example before should convince anybody of the correctness of the translation scheme G'' :



The dashed arrows trace the frontiers of the source and destination trees, and show the cancellation of the leading zeroes from the destination one.

The source grammar G''_s is not ambiguous either. It is evident that also the alternative rules that expand nonterminal N could be turned into right-linear form. This however would change the grammar more than necessary.

The translation can be modeled by the following regular translation scheme:

$$\left(\begin{array}{c} 0 \\ \varepsilon \end{array} \right)^* \frac{1}{1} \left(\begin{array}{c} 0 \\ 0 \end{array} \mid \begin{array}{c} 1 \\ 1 \end{array} \right)^*$$

and the destination language is $1(0 \mid 1)^*$, regular as expected. Of course, the source language consists of all the bit strings that contain at least one bit 1.

- (c) First (as suggested) we had better make explicit the source and destination languages L_s and L_d of translation (b), since the surjective and injective properties² depend on them. These languages, both regular as said at point (b), can be quickly obtained from the regular translation scheme of point (b):

$$L_s = 0^* 1 (0 \mid 1)^* \quad L_d = 1 (0 \mid 1)^*$$

Both languages L_s and L_d are a proper subset of the universal language, i.e., $L_s, L_d \subsetneq (0 \mid 1)^*$, and language L_d is a proper subset of L_s , i.e., $L_d \subsetneq L_s$.

Translation (b) is *surely surjective*: each destination bit string has at least one (actually more than one) source bit string as its counterimage; more precisely, it has all the source strings equal to itself plus a (possibly empty) prefix composed only of zeroes. In symbols: each destination string $1 (0 \mid 1)^* = s$ is the image of all the (infinitely many) source strings $0^* s$.

For the same reason, translation (b) is *certainly not injective*: all the source bit strings that differ only by a prefix composed of zeroes, are translated into the same destination bit string. In symbols: all the (infinitely many) source strings $0^* \underbrace{1 (0 \mid 1)^*}_s = 0^* s$ are translated into the same destination string s .

In summary, the translation of point (b) is surjective but not injective, and consequently, it is not bijective (one-to-one) either.

For completeness, here are the two translations (a) and (b) written in fractional form:

$$G' \left\{ \begin{array}{l} S \rightarrow \frac{\varepsilon}{e} S \frac{0}{0} \mid N \frac{1}{1} \\ N \rightarrow N \frac{0}{0} \mid N \frac{1}{1} \mid \frac{\varepsilon}{\varepsilon} \end{array} \right.$$

$$G'' \left\{ \begin{array}{l} S \rightarrow \frac{0}{\varepsilon} S \mid \frac{1}{1} N \\ N \rightarrow N \frac{0}{0} \mid N \frac{1}{1} \mid \frac{\varepsilon}{\varepsilon} \end{array} \right.$$

These forms are just a notational change of the previous representations, of course.

Similarly, the two translation grammars could be rewritten in the equivalent brace notation, as follows:

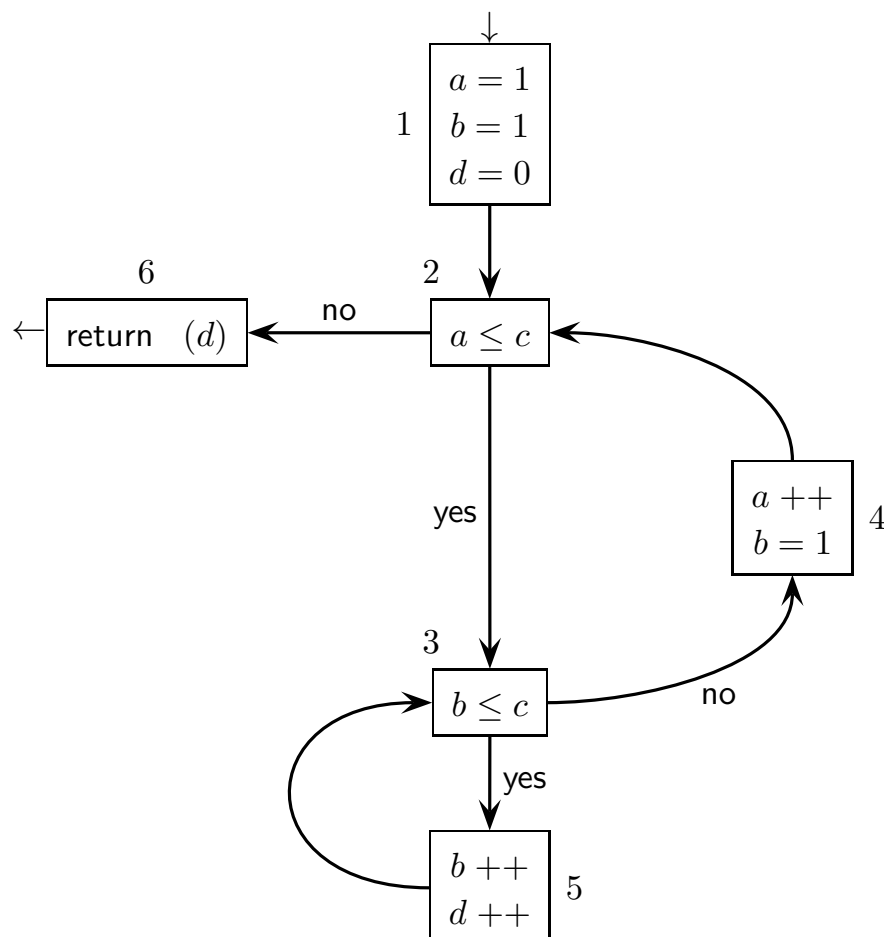
$$G' \left\{ \begin{array}{l} S \rightarrow \{e\} S 0 \{0\} \mid N 1 \{1\} \\ N \rightarrow N 0 \{0\} \mid N 1 \{1\} \mid \varepsilon \end{array} \right.$$

$$G'' \left\{ \begin{array}{l} S \rightarrow 0 S \mid 1 \{1\} N \\ N \rightarrow N 0 \{0\} \mid N 1 \{1\} \mid \varepsilon \end{array} \right.$$

Notice that when the destination grammar contains the empty string $\{\varepsilon\}$, for simplicity we drop it, e.g., writing $\frac{0}{\varepsilon} = 0 \{ \varepsilon \}$ and $\frac{\varepsilon}{\varepsilon} = \varepsilon \{ \varepsilon \}$ is shortened to 0 and ε , respectively. Similarly, instead of writing $\frac{\varepsilon}{e} = \varepsilon \{ e \}$, we shorten to $\{e\}$.

²From basic mathematics: by definition a function $f: D \rightarrow I$ is *surjective* if every image element $y \in I$ has at least one counterimage $x \in D$, i.e., $\forall y \in I \exists x \in D$ such that $y = f(x)$, and it is *injective* if different domain elements have different images, i.e., $\forall x_1, x_2 \in D \quad x_1 \neq x_2 \Rightarrow f(x_1) \neq f(x_2)$; and the function is *bijective* or *one-to-one* if it is both surjective and injective.

2. Consider the following Control Flow Graph (*CFG*) of a program, with some nodes that contain multiple assignments (input node 1 and output node 6):

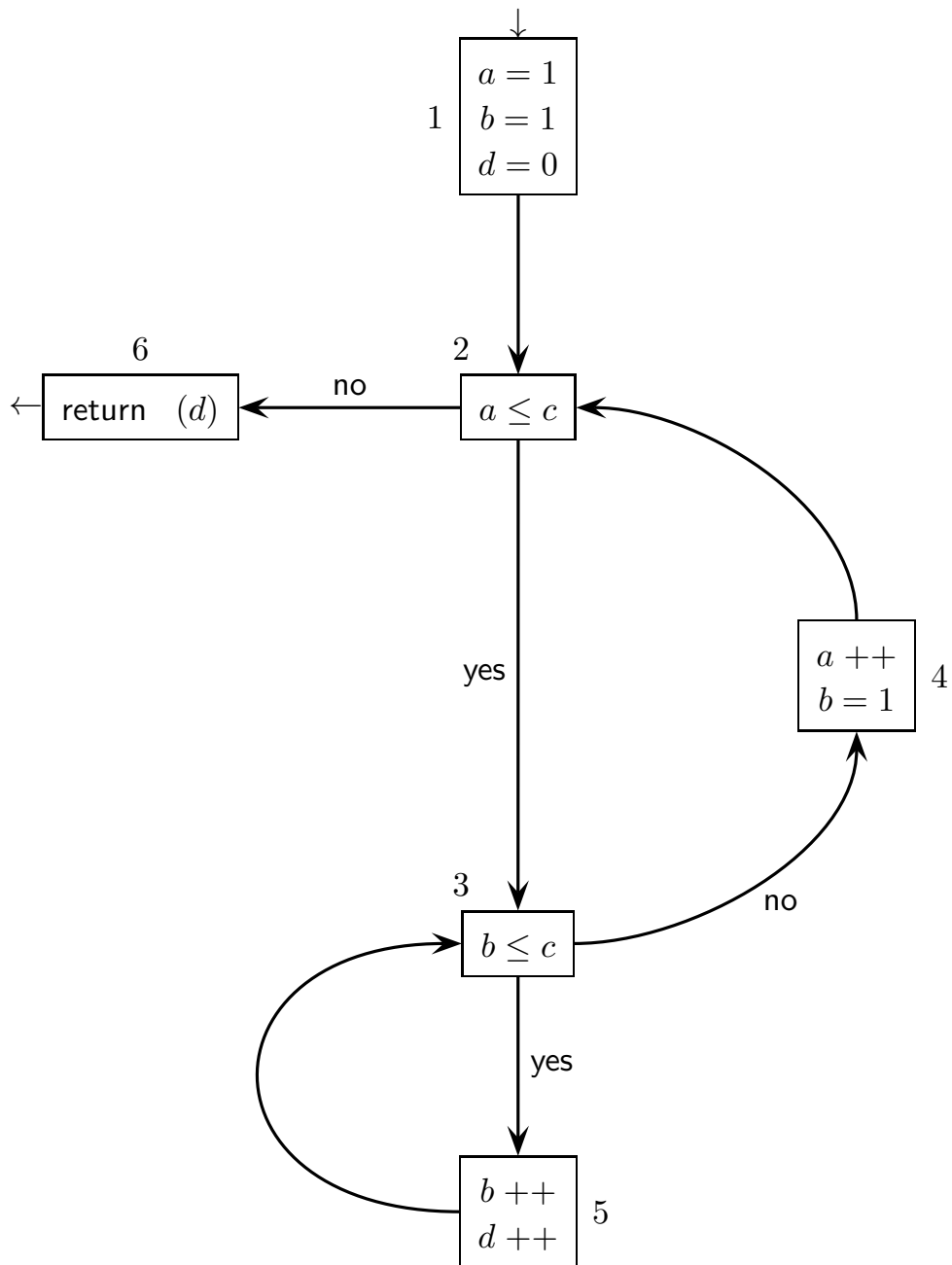


This program has three variables a , b and d , and an input parameter c . All of them are integers ≥ 0 .

Answer the following questions:

- Find a regular expression R , over the node name alphabet, that describes the execution traces of the program, ignoring any semantic restriction.
- Directly identify the variables live at the node inputs of the *CFG* and write them on the *CFG* (use the figure prepared on the next page).
- Write the system of flow equations for the live variables of the *CFG*, and iteratively solve the system (use the tables prepared on the next pages). Verify that the obtained solution coincides with that found at point (b).
- (optional) Determine the value d returned by the program as a function of the input parameter c . Based on this function, refine the regular expression R found at point (a) into a language expression over the node name alphabet that exactly defines the program execution traces. Is the language of the execution traces still regular? Adequately justify your answer.

please write here the variables live at the node inputs



tables of definitions and usages at the nodes

<i>node</i>	<i>defined</i>	<i>node</i>	<i>used</i>
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	

system of data-flow equations

<i>node</i>	<i>in equations</i>	<i>out equations</i>
1		
2		
3		
4		
5		
6		

iterative solution table of the system of data-flow equations
 (the number of columns is not significant and is more than sufficient to solve the system)

	<i>initialization</i>		1		2		3		4		5	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1												
2												
3												
4												
5												
6												

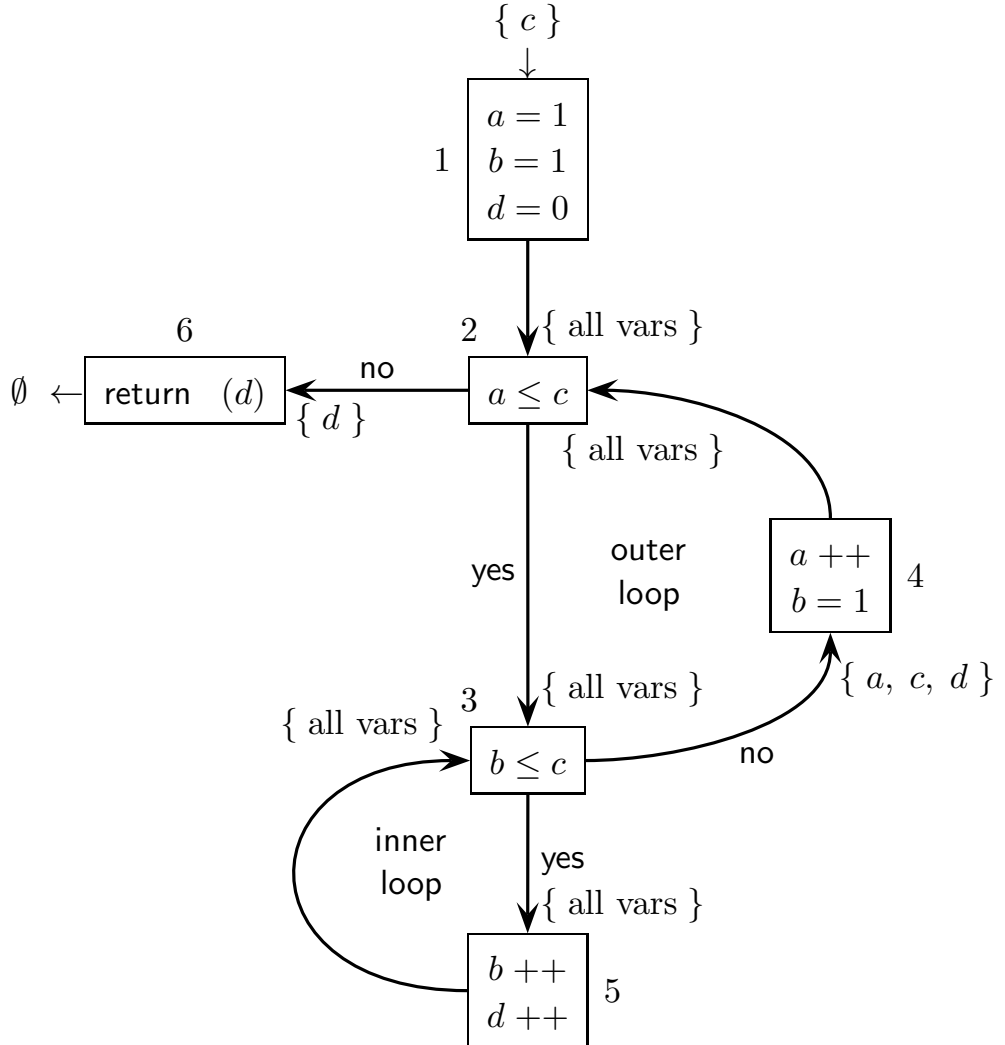
Solution

- (a) Here are two regular expressions R , obtained intuitively, for the program traces:

$$R = 1 \overbrace{\left(2 \underbrace{(3 \ 5)^*}_{\text{inner loop}} 3 \ 4 \right)^*}_{\text{outer loop}} 2 \ 6 = 1 \ 2 \left(3 \ (5 \ 3)^* 4 \ 2 \right)^* 6$$

The *CFG* consists of two nested loops, namely the inner loop (3, 5) and the outer loop (2, (inner loop), 3, 4), with some prologue and epilogue, so finding the regular expression R is simple. One could quickly obtain a regular expression R by node elimination as well, the same as above or an equivalent one.

- (b) Here are the live variables of the *CFG*, obtained directly from their definition:



The input parameter c is live on the input of the initial node 1. All the variables are live on most nodes, with few exceptions. Variable b is not live on node 4, as it is redefined yet unused therein. Only variable d is live on the final node 6, as it is the only one used therein. The auto-increment operator $++$ causes the variable to be both used and defined, since e.g. $a ++$ is equivalent to $a = a + 1$.

(c) Here is the flow equation method:

tables of definitions and usages at the nodes

<i>node</i>	<i>defined</i>	<i>node</i>	<i>used</i>
1	$a\ b\ d$	1	—
2	—	2	$a\ c$
3	—	3	$b\ c$
4	$a\ b$	4	a
5	$b\ d$	5	$b\ d$
6	—	6	d

system of data-flow equations

<i>node</i>	<i>in equations</i>	<i>out equations</i>
1	$in(1) = out(1) - \{ a\ b\ d \}$	$out(1) = in(2)$
2	$in(2) = \{ a\ c \} \cup (out(2) - \emptyset)$	$out(2) = in(3) \cup in(6)$
3	$in(3) = \{ b\ c \} \cup (out(3) - \emptyset)$	$out(3) = in(4) \cup in(5)$
4	$in(4) = \{ a \} \cup (out(4) - \{ a\ b \})$	$out(4) = in(2)$
5	$in(5) = \{ b\ d \} \cup (out(5) - \{ b\ d \})$	$out(5) = in(3)$
6	$in(6) = \{ d \} \cup (out(6) - \emptyset)$	$out(6) = \emptyset$

Some flow equations can be more or less simplified, namely:

<i>node</i>	<i>in equations</i>	<i>out equations</i>
1	$in(1) = \{ c \}$	$out(1) = in(2)$
2	$in(2) = \{ a\ c \} \cup out(2)$	$out(2) = in(3) \cup in(6)$
3	$in(3) = \{ b\ c \} \cup out(3)$	$out(3) = in(4) \cup in(5)$
4	$in(4) = \{ a \} \cup (out(4) - \{ a\ b \})$	$out(4) = in(2)$
5	$in(5) = \{ b\ d \} \cup (out(5) - \{ b\ d \})$	$out(5) = in(3)$
6	$in(6) = \{ d \}$	$out(6) = \emptyset$

Notice that c is an input parameter and is used in the program, thus it is the only variable that can be live at the input of the initial node 1.

iterative solution table of the system of data-flow equations using the non-simplified version
(the number of columns is not significant)

	<i>initialization</i>		1		2		3		4		5	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1	$\emptyset$	$\emptyset$	$a\ c$	c	$a\ b\ c\ d$	c	$a\ b\ c\ d$					
2	$\emptyset$	$a\ c$	$b\ c\ d$	$a\ b\ c\ d$	$a\ b\ c\ d$	$a\ b\ c\ d$	$a\ b\ c\ d$					
3	$\emptyset$	$b\ c$	$a\ b\ d$	$a\ b\ c\ d$	$a\ b\ c\ d$	$a\ b\ c\ d$	$a\ b\ c\ d$					
4	$\emptyset$	a	$a\ c$	$a\ c$	$a\ b\ c\ d$	$a\ c\ d$	$a\ b\ c\ d$					
5	$\emptyset$	$b\ d$	$b\ c$	$b\ c\ d$	$a\ b\ c\ d$	$a\ b\ c\ d$	$a\ b\ c\ d$					
6	$\emptyset$	d	$\emptyset$	d	$\emptyset$	d	$\emptyset$					

The solution converges in two steps only, since the columns *out* at iterations 2 and 3 coincide.
Column *in* at step 2 provides the live variables at the node inputs. The solution is identical to the direct one.

- (d) The input-output relation of this program is that the algorithm computes the square of the input parameter c , i.e., $d = c^2$, as follows:

$$\begin{array}{c}
 \text{initial} \\
 \text{value} \\
 d = 0 + \overbrace{\underbrace{\frac{1}{b=1} + \frac{1}{b=2} + \dots + \frac{1}{b=c}}_{\text{inner loop} - c \text{ times}} + \underbrace{\text{idem} + \dots + \text{idem}}_{\substack{\text{outer loop} - c \text{ times} \\ a=1 \quad a=2 \quad a=c}}} = c \times c = c^2
 \end{array}$$

Said synthetically, the program consists of two nested independent loops, each of which iterates exactly c times per activation. They can be thought of as counter loops, ruled by variables a (outer) and b (inner), respectively. Thus the total number of increments $d++$ of the output variable d , which are executed in the inner loop, is the product of the numbers of loop iterations, i.e., $c \times c = c^2$.

A language formulation that keeps the program semantics into consideration is a sort of refinement R_{ref} of the regular expression R , namely:

$$R_{ref} = 1 \left(2 \underbrace{(3 \ 5)^h}_{\text{inner loop}} 3 \ 4 \right)^h 2 \ 6 \quad \text{--- for any integer } h \geq 0$$

where it holds $h = c \geq 0$, the input parameter. Said differently, one can see that the program semantics imposes to derive the two star operators ‘ $*$ ’ — both the inner one and the outer one — by choosing an exponent $h = c$ equal for both.

Clearly the formulation R_{ref} is not a regular expression. The refined language $L(R_{ref})$ of the execution traces of the *CFG* is not regular either, as it has two equal exponents that cannot be granted by two independent iteration operators. The regular expression R of point (a) generates a language $L(R)$ that is a superset of the refined language $L(R_{ref})$ of the program execution traces, i.e., $L(R) \supsetneq L(R_{ref})$. In fact, language $L(R)$ also contains all the execution traces that correspond to different numbers of iterations of the two program loops. Such traces could never be observed during a real execution of the program.